

SISTEMAS DE GESTIÓN

CASO PRACTICO I UD5



ALUMNO CESUR

25/26

Alejandro Muñoz de la Sierra

PROFESOR

Óscar González Núñez

INTRODUCCION

Empezamos este documento detallando cómo hemos montado desde cero el módulo personalizado "Gestión Académica Universitaria", utilizando el *framework* del ERP Odoo en su versión 19. La idea principal de este caso práctico no era hacer el típico ejercicio de clase, sino aplicar de verdad todo lo que hemos visto sobre arquitectura de software de empresa, tocando desde la base de datos (con los modelos del ORM) hasta lo que ve el usuario final (las vistas y reportes en XML).

El supuesto nos pedía informatizar un departamento educativo manejando dos bloques clave: los Cursos académicos y las Asignaturas que se ofertan. Para sacarlo adelante, hemos hecho el ciclo de desarrollo completo de principio a fin. Empezamos generando el andamiaje inicial con *scaffold*, luego programamos los modelos y su relación cruzada (el famoso *Many2many*), inyectamos datos de prueba automatizados (*Demo Data*) y rematamos haciendo que el sistema genere un informe en PDF bastante apañado usando la tecnología QWeb.

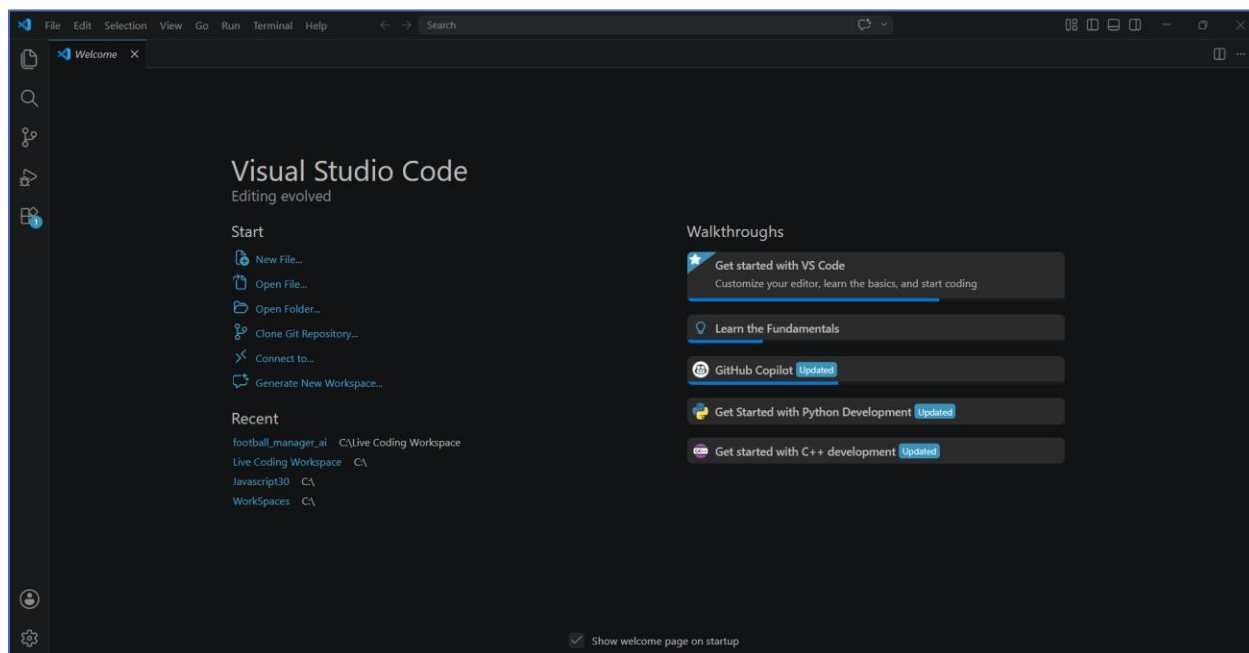
Pero la verdad es que esto ha ido mucho más allá de simplemente crear código. Esta práctica nos ha exigido hacer de administradores de sistemas y pelear con la integración. Hemos tenido que conectar Visual Studio Code con el motor de Odoo, gestionar PostgreSQL por debajo y apagar "incendios" técnicos en tiempo real, sintiéndonos por unas semanas como si estuviéramos trabajando de consultores o implantadores en un entorno corporativo real.

PREPARACIÓN DEL ENTORNO DE DESARROLLO

Para empezar a desarrollar **módulos** a medida en **Odoo**, lo primero que tuvimos que hacer fue cambiar un poco el "chip" y **apartar** temporalmente **Eclipse** y los IDEs centrados en **Java**. Odoo tiene una arquitectura multicapa muy suya, así que **necesitábamos** preparar un **entorno** que se llevara bien tanto con **Python** (para toda la **lógica** de negocio) como con **XML**, **HTML** y **JS** para la parte de las **vistas**. Los pasos que hemos seguido para dejar nuestro entorno de trabajo listo han sido estos:

1.1. Instalación del Entorno de Desarrollo Integrado (IDE)

Nos decantamos por Visual Studio Code (**VS Code**). La verdad es que es un entorno súper ligero y modular, y es seguramente la opción más cómoda cuando tienes que ir saltando entre varios lenguajes a la vez dentro del ecosistema de Odoo.

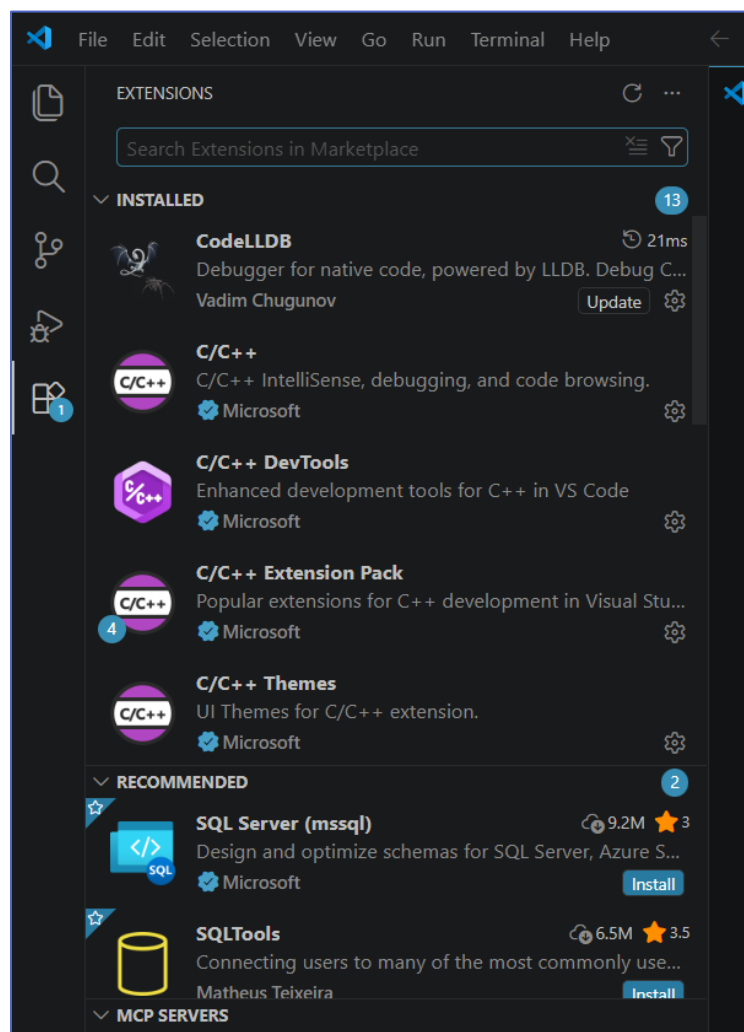


1.2. Configuración del Lenguaje Base (Python)

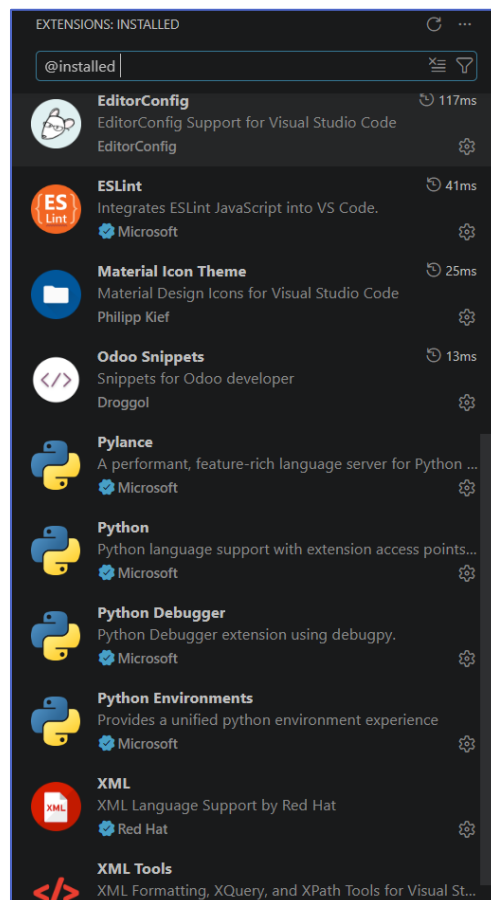
Como **Odoo utiliza Python** para **programar** las **clases**, los **modelos** y toda la **lógica** que hay por debajo, el primer paso lógico fue instalar el intérprete en nuestras máquinas. Una vez hecho esto, **configuramos pip** (el **gestor de paquetes de Python**) para poder **instalar** las **dependencias** que exige el motor de Odoo. Por ejemplo, tirando comandos por consola como `pip install Jinja2` o instalando de golpe todo el fichero `requirements.txt`.

1.3. Instalación de Extensiones y Plugins en VS Code

Para no tener que escribir todo el código "a pelo" y tener algo de ayuda con el autocompletado y los chivatos de errores (*linting*), nos bajamos unas cuantas **extensiones** clave **desde el Marketplace** de VS Code que nos han facilitado bastante la vida:



- **Python (ms-python.python) y Pylance:** Básicas para poder escribir la lógica de negocio, tener un buen autocompletado y poder depurar (*debuggear*) cuando algo falla. Eso sí, una vez instaladas hay que acordarse de seleccionar el intérprete de Python correcto (abriendo la paleta con *Ctrl+Shift+P > Python: Select Interpreter*).
- **XML Tools:** Súper necesaria para crear las vistas (.xml), montar los menús y definir las acciones de Odoo sin liarte con la estructura de las etiquetas.
- **ESLint:** Nos viene genial para que nos marque los fallos de sintaxis cuando nos toca programar en el *frontend* con JavaScript.
- **EditorConfig for VS Code:** Para mantener un estilo de código limpio y consistente, sobre todo si trabajas en equipo.
- **Odoo Snippets:** Esta la instalamos por recomendación y ahorra muchísimo tiempo. Te permite meter bloques de código ya predefinidos para Odoo (como modelos enteros, campos o vistas) escribiendo solo un par de letras.



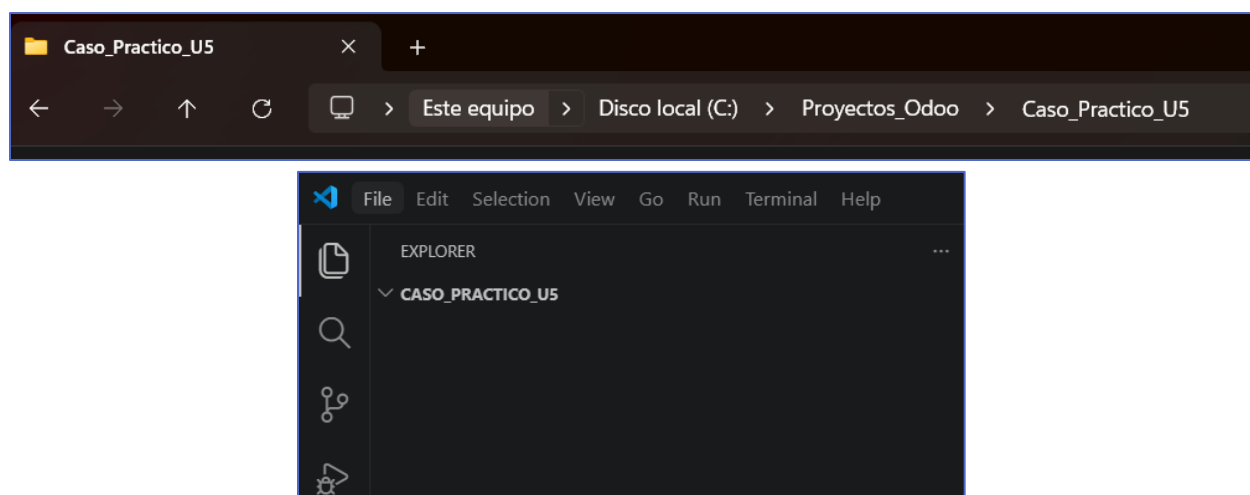
CREACIÓN DE LA ESTRUCTURA BASE DEL MÓDULO (SCAFFOLD) Y ESPACIO DE TRABAJO

Cuando nos pusimos con nuestro primer desarrollo real en Odoo, tuvimos clara una cosa desde el principio: **no íbamos a mezclar nuestro código con los archivos del núcleo del programa**. Trabajar directamente dentro de las carpetas donde se instala Odoo (server/odoo/addons) es jugársela bastante. Es una mala práctica que luego te complica la vida con el control de versiones, corres el riesgo de romper el propio ERP y, además, a la hora de entregar la práctica es un caos intentar separar qué archivos son tuyos y cuáles son del sistema. Así que **decidimos montarnos nuestro propio espacio de trabajo totalmente aislado**.

2.1. Preparación del Directorio de Trabajo

Primero nos creamos una carpeta vacía en el ordenador que fuera exclusivamente para este proyecto. Nos hicimos una ruta tipo

C:\Proyectos_Odoo\Caso_Practico_U5. Luego, desde Visual Studio Code, abrimos directamente esa carpeta (**File > Open Folder...**). De esta manera, todo lo que vayamos programando se queda ahí dentro, limpio, encapsulado y súper fácil de comprimir luego en un ZIP para enviárselo al profesor sin arrastrar basura de otros sitios.



2.2. Generación del Esqueleto con scaffold y Gestión del Entorno de Ejecución

En **Odoo**, **montar un módulo** desde cero implica **crear una jerarquía de carpetas** (models, views, security, etc.) y unos archivos que tienen que ser súper exactos. Para no tener que hacer todo esto a mano y arriesgarnos a liarla con algún nombre, la documentación oficial aconseja **usar el comando interno scaffold**. Esto básicamente te escupe un "esqueleto" ya preparado para que empieces a programar encima.

El problema vino cuando intentamos lanzar este comando en Windows desde la terminal de PowerShell en VS Code. Nos topamos con un par de detalles técnicos bastante curiosos que tuvimos que esquivar para que funcionara:

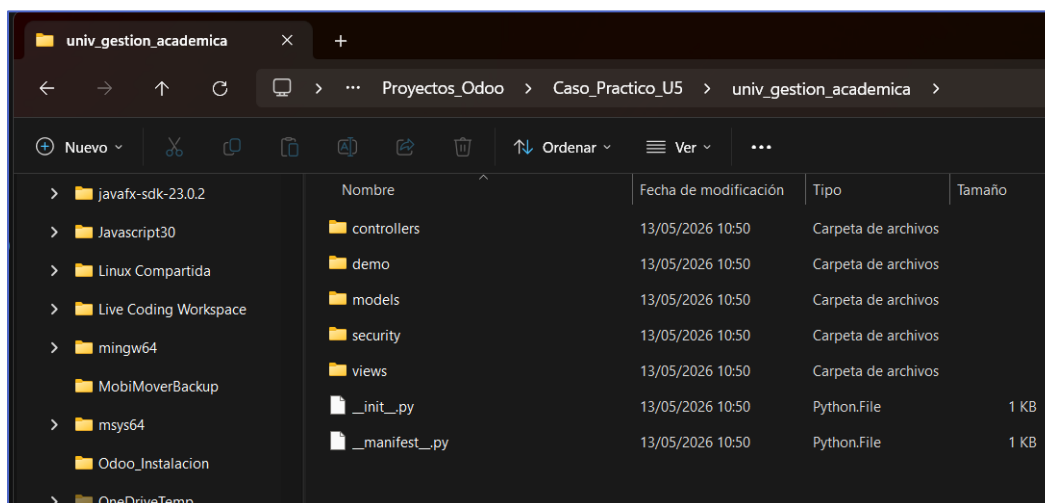
- **Gestión de dependencias (La pelea con Python):** Cuando intentamos tirar el comando usando el Python global que teníamos instalado en el sistema, nos saltó una cascada de errores (el típico `ModuleNotFoundError: No module named 'babel'`). Nos dimos cuenta de que **Odoo necesita un ecosistema de librerías bastante complejo** (jinja2, werkzeug, psycopg2) que un Python limpio, lógicamente, no tiene. En vez de volvernos locos intentando compilar e instalar todo eso a mano con pip en Windows (que casi siempre da algún dolor de cabeza), tiramos por una decisión más estratégica: **usar directamente el intérprete de Python embebido que ya trae la propia instalación de Odoo, que viene con el 100% de los requisitos listos para funcionar.**
- **El truco de PowerShell (El operador de llamada):** Como las rutas de instalación en Windows suelen tener espacios en blanco (por el mítico Program Files), hay que envolverlas entre comillas dobles sí o sí. Pero claro, PowerShell lee esas comillas y se piensa que es un texto normal y corriente (un *String*), así que nos soltaba un error de "Token inesperado" y pasaba de ejecutar nada. Para arreglarlo, **tuvimos que plantarle el operador de llamada (&) justo al principio.** Esto le avisa a la terminal de que evalúe esa cadena de texto como un comando ejecutable real.

Una vez aclarado todo esto, apuntamos nuestra terminal hacia nuestro espacio de trabajo aislado y lanzamos este comando:

Ejecutamos el Python embebido de Odoo, llamamos al motor odoo-bin, invocamos la herramienta scaffold para el módulo 'univ_gestion_academica' y establecemos como destino nuestra carpeta de desarrollo independiente.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Proyectos_Odoo\Caso_Practico_U5> & "C:\Program Files\Odoo 19.0.20251129\python\python.exe" "C:\Program Files\Odoo 19.0.20251129\server\odoo-bin" scaffold univ_gestion_academica "C:\Proyectos_Odoo\Caso_Practico_U5"
```

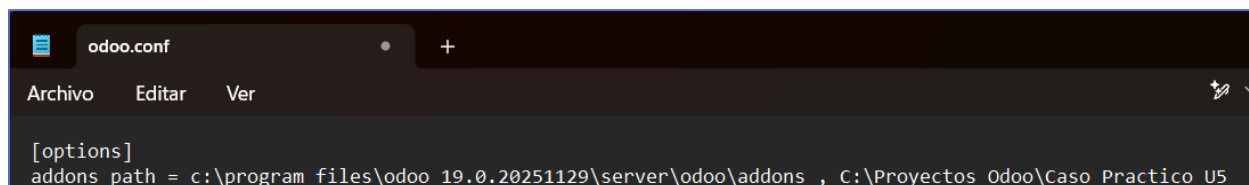
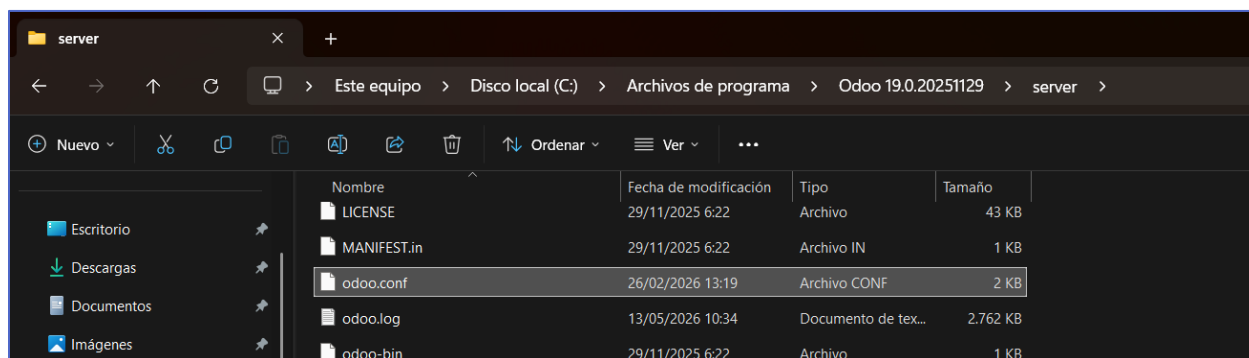
Fue darle a intro y magia: nos generó al instante y sin un solo fallo toda la estructura de directorios (__init__.py, __manifest__.py, la carpeta de models, las views, etc.) justo dentro de Caso_Practico_U5. Nos dejó una base totalmente limpia y encapsulada, lista para ponernos a picar de verdad la lógica de nuestra universidad.



2.3. Configuración del addons_path (Conectando Odoo con nuestro código)

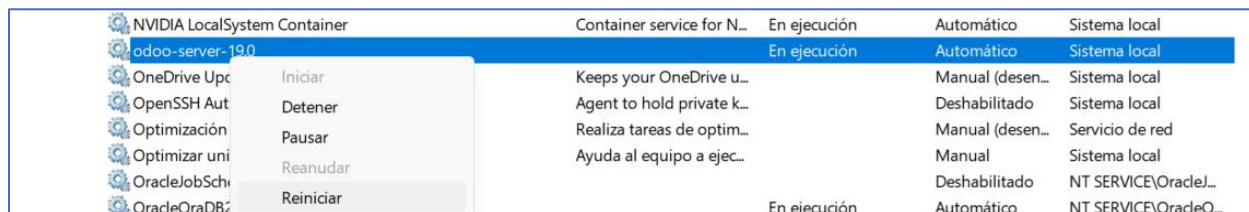
Aquí nos topamos con un detalle importante. Como **nuestro módulo** ahora "vivía" en una carpeta externa (y no en las que Odoo trae por defecto), el servidor estaba totalmente ciego; no sabía que nuestro trabajo existía. Tuvimos que ir a **decirle a Odoo dónde tenía que ir a buscarlo**.

Para arreglarlo, buscamos el archivo de configuración del servidor (que normalmente se llama **odoo.conf** y suele estar en la carpeta de instalación). Lo abrimos, buscamos la línea que pone **addons_path** y simplemente le añadimos la ruta de nuestra nueva carpeta, separada por una coma:



Con este paso tan tonto, nos aseguramos de que **cada vez que reiniciemos el servidor, Odoo escanee nuestra carpeta y detecte sin problema el nuevo módulo** de la universidad.

Importante recordar que una vez hecho este cambio, necesitamos reiniciar el servicio de Odoo o no reconocerá nuestra carpeta.



2.4. Personalización del Archivo `manifest.py` (Definición e Identidad)

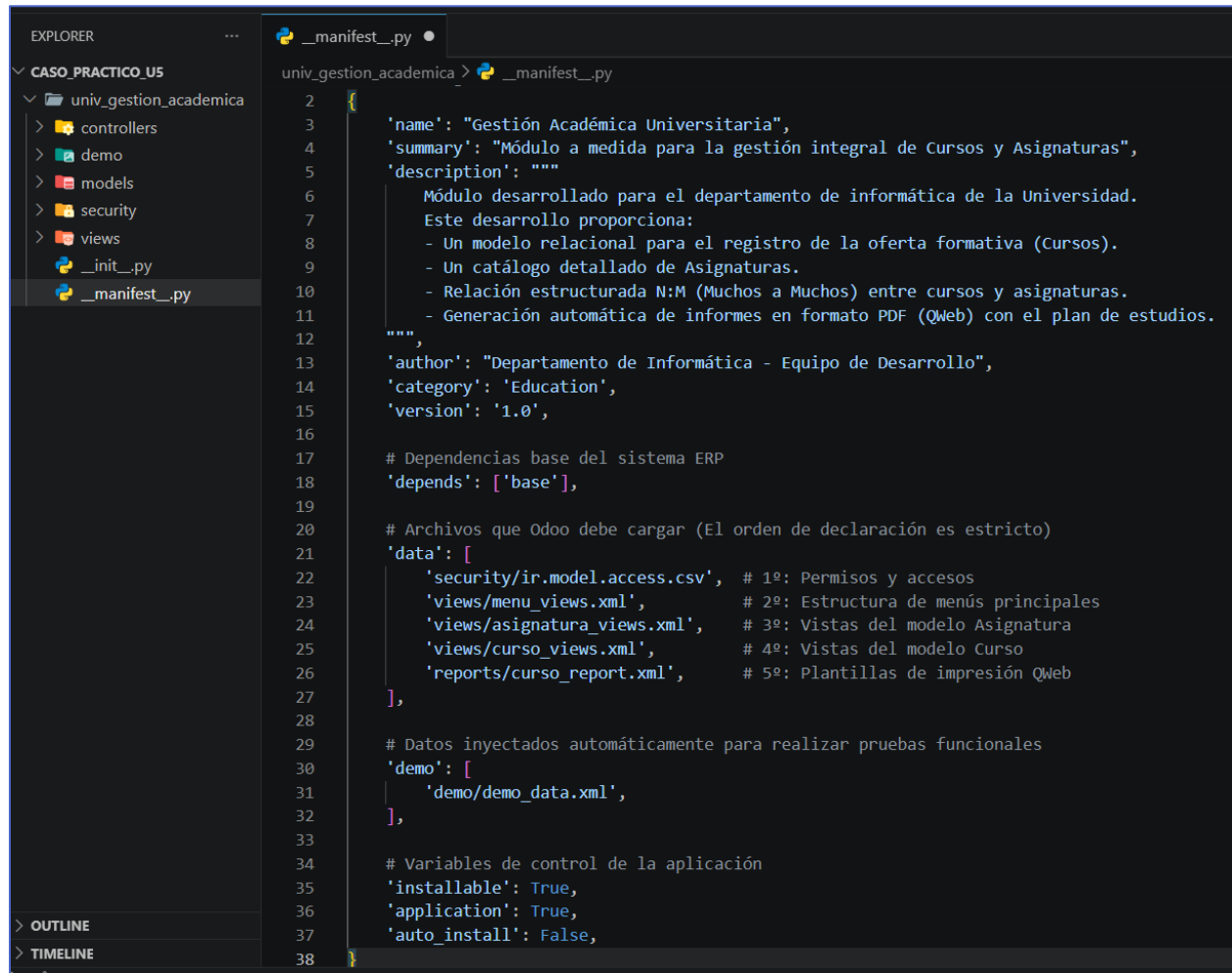
Después de que el comando **scaffold** nos **generara toda la estructura vacía**, **tocaba empezar a llenar en los archivos**. El más importante de todos, y el primero que tuvimos que tocar, fue el `__manifest__.py`.

Básicamente, este *script* de Python es como el "DNI" de nuestro módulo. Antes de hacer nada, el motor de **Odoo lee este archivo** para enterarse de qué va el desarrollo, qué **dependencias necesita** para poder funcionar y, sobre todo, **qué archivos de interfaz (XML) tiene que inyectar** luego en la base de datos.

Empezamos limpiando todo el texto y los comentarios genéricos que nos había metido el scaffold por defecto, y configuramos los metadatos reales que nos pedía el enunciado de la Universidad. Al configurarlo, nos dimos cuenta de que hay dos diccionarios que son absolutamente clave para que el módulo no se rompa:

- **'depends'**: Aquí le indicamos al sistema que nuestro desarrollo necesita apoyarse sí o sí en el módulo 'base' de Odoo para poder arrancar.
- **'data'**: Aquí es donde tenemos que **listar todos los archivos XML** que vamos a ir creando para las vistas. Lo más curioso que aprendimos (y que nos costó algún que otro error al principio) es que el orden en este *array* es súper crítico. Resulta que Odoo lee los archivos de forma estrictamente secuencial, de arriba a abajo. Si te despistas y pones el archivo de las vistas antes que el archivo de seguridad, el sistema intenta dibujar unas pantallas a las que el usuario todavía no tiene permiso para entrar, y la instalación del módulo crashea directamente.

Teniendo todo esto en cuenta para no liarla con el orden de carga, dejamos nuestro `__manifest__.py` definitivo configurado de la siguiente manera:



```
2  {
3      'name': "Gestión Académica Universitaria",
4      'summary': "Módulo a medida para la gestión integral de Cursos y Asignaturas",
5      'description': """
6          Módulo desarrollado para el departamento de informática de la Universidad.
7          Este desarrollo proporciona:
8          - Un modelo relacional para el registro de la oferta formativa (Cursos).
9          - Un catálogo detallado de Asignaturas.
10         - Relación estructurada N:M (Muchos a Muchos) entre cursos y asignaturas.
11         - Generación automática de informes en formato PDF (Qweb) con el plan de estudios.
12     """,
13     'author': "Departamento de Informática - Equipo de Desarrollo",
14     'category': 'Education',
15     'version': '1.0',
16
17     # Dependencias base del sistema ERP
18     'depends': ['base'],
19
20     # Archivos que Odoo debe cargar (El orden de declaración es estricto)
21     'data': [
22         'security/ir.model.access.csv', # 1º: Permisos y accesos
23         'views/menu_views.xml',        # 2º: Estructura de menús principales
24         'views/asignatura_views.xml',   # 3º: Vistas del modelo Asignatura
25         'views/curso_views.xml',        # 4º: Vistas del modelo Curso
26         'reports/curso_report.xml',     # 5º: Plantillas de impresión Qweb
27     ],
28
29     # Datos inyectados automáticamente para realizar pruebas funcionales
30     'demo': [
31         'demo/demo_data.xml',
32     ],
33
34     # Variables de control de la aplicación
35     'installable': True,
36     'application': True,
37     'auto_install': False,
38 }
```

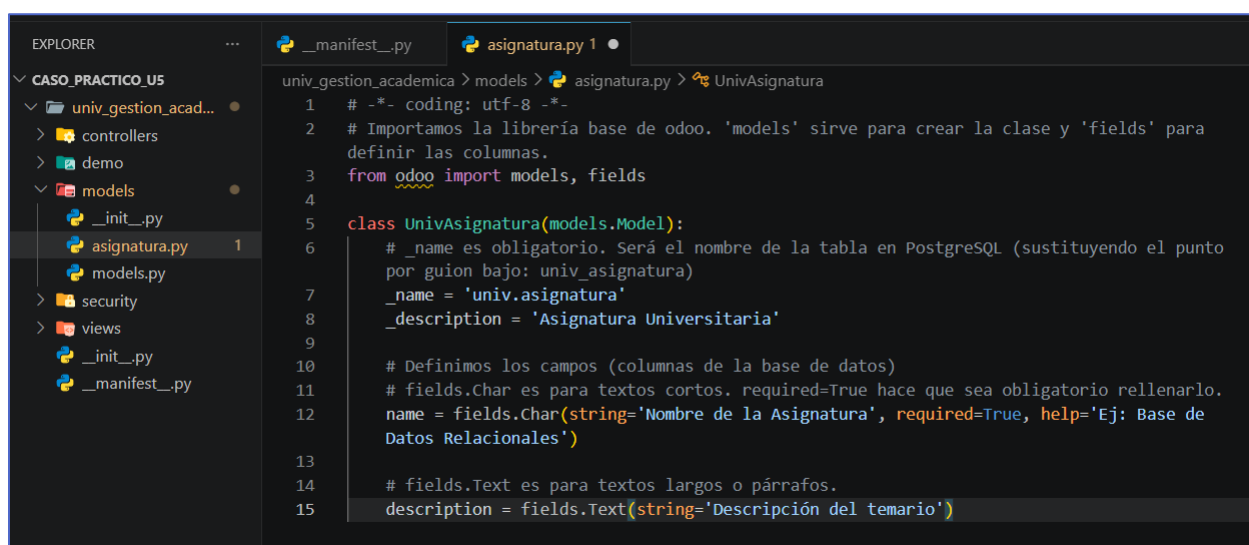
DESARROLLO DE LA LÓGICA DE DATOS (MODELOS EN PYTHON)

Viniendo de Java estamos muy acostumbrados a pensar en Clases. En Odoo con Python la idea es la misma, solo que las **clases heredan de** una principal llamada **models.Model**. La magia de esto es que, **cada clase** que programas aquí, el ORM de **Odoo la transforma sola en una tabla** dentro de la base de datos **PostgreSQL**.

Para no hacer un archivo gigante, creamos **dos scripts separados** dentro de la **carpeta models**.

3.1. Archivo: models/asignatura.py

Empezamos por la parte fácil, la entidad asignatura:

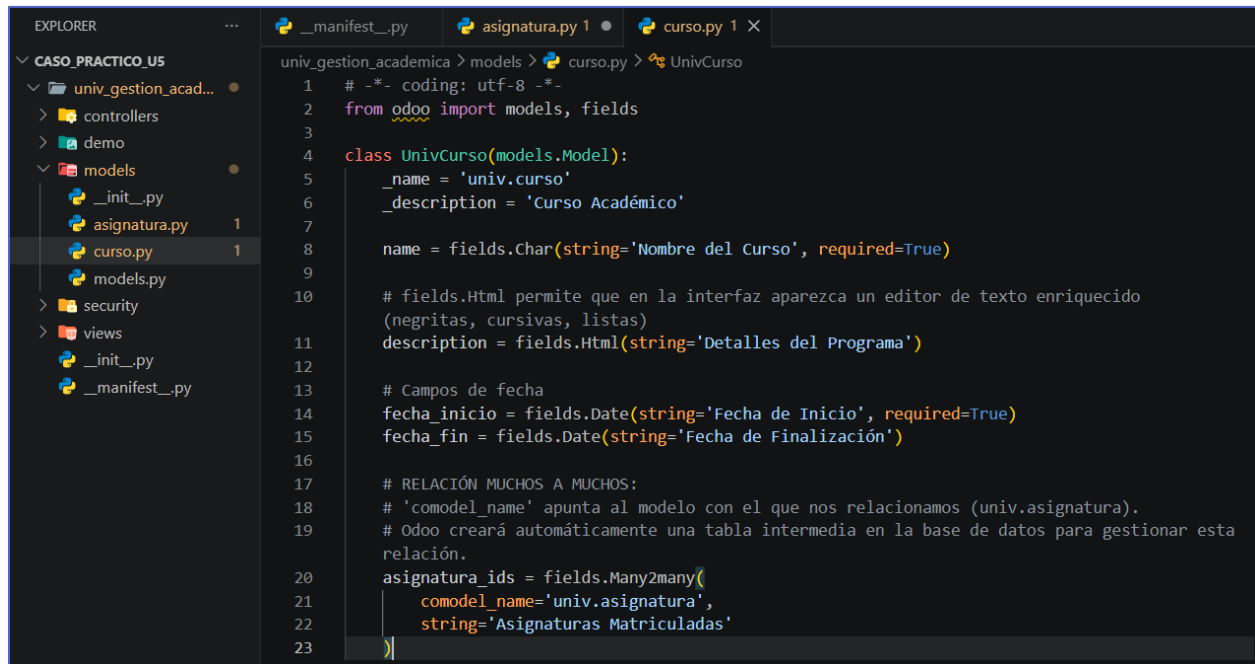


```

1  # -*- coding: utf-8 -*-
2  # Importamos la librería base de odoo. 'models' sirve para crear la clase y 'fields' para
   definir las columnas.
3  from odoo import models, fields
4
5  class UnivAsignatura(models.Model):
6      # _name es obligatorio. Será el nombre de la tabla en PostgreSQL (sustituyendo el punto
   por guion bajo: univ_asignatura)
7      _name = 'univ.asignatura'
8      _description = 'Asignatura Universitaria'
9
10     # Definimos los campos (columnas de la base de datos)
11     # fields.Char es para textos cortos. required=True hace que sea obligatorio rellenarlo.
12     name = fields.Char(string='Nombre de la Asignatura', required=True, help='Ej: Base de
   Datos Relacionales')
13
14     # fields.Text es para textos largos o párrafos.
15     description = fields.Text(string='Descripción del temario')
  
```

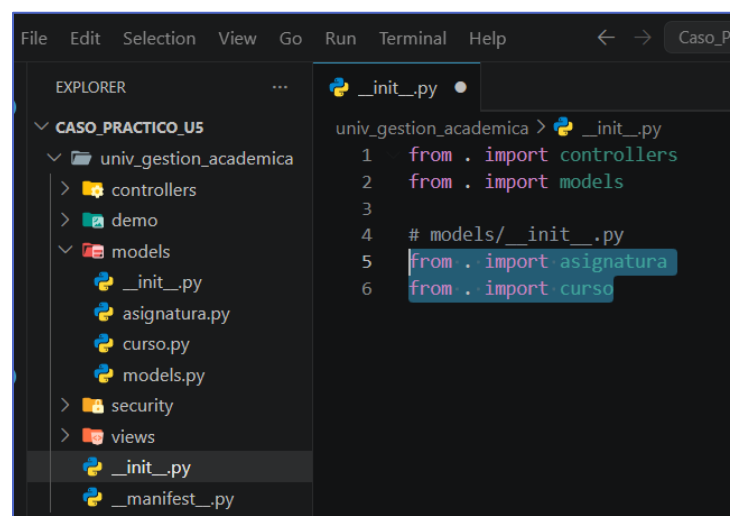
3.2. Archivo: models/curso.py

El curso era más complejo de hacer. Aquí nos tocó investigar cómo hacer que un curso tuviera varias asignaturas dentro y que esas asignaturas pudieran pertenecer a varios cursos. La solución pasaba por usar un campo Many2many (Muchos a Muchos).



```
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields
3
4  class UnivCurso(models.Model):
5      _name = 'univ.curso'
6      _description = 'Curso Académico'
7
8      name = fields.Char(string='Nombre del Curso', required=True)
9
10     # fields.Html permite que en la interfaz aparezca un editor de texto enriquecido
11     # (negritas, cursivas, listas)
12     description = fields.Html(string='Detalles del Programa')
13
14     # Campos de fecha
15     fecha_inicio = fields.Date(string='Fecha de Inicio', required=True)
16     fecha_fin = fields.Date(string='Fecha de Finalización')
17
18     # RELACIÓN MUCHOS A MUCHOS:
19     # 'comodel_name' apunta al modelo con el que nos relacionamos (univ.asignatura).
20     # Odoo creará automáticamente una tabla intermedia en la base de datos para gestionar esta
21     # relación.
22     asignatura_ids = fields.Many2many(
23         comodel_name='univ.asignatura',
24         string='Asignaturas Matriculadas'
```

Paso Crítico: Una cosa que aprendimos a base de fallos es que, para que Odoo se entere de que estos archivos existen, **hay que importarlos en el archivo models/__init__.py**. Si te saltas esto, Odoo arranca pero no crea las tablas.



```
1  from . import controllers
2  from . import models
3
4  # models/__init__.py
5  from . import asignatura
6  from . import curso
```

DISEÑO DE LA INTERFAZ Y VISTAS (ARCHIVOS XML)

Vale, ya teníamos los modelos con los datos, pero necesitábamos pantallas para poder interactuar con ellos. **En Odoo las vistas se montan inyectando código XML directamente.**

4.1. Creación de Menús y Acciones (views/menu_views.xml)

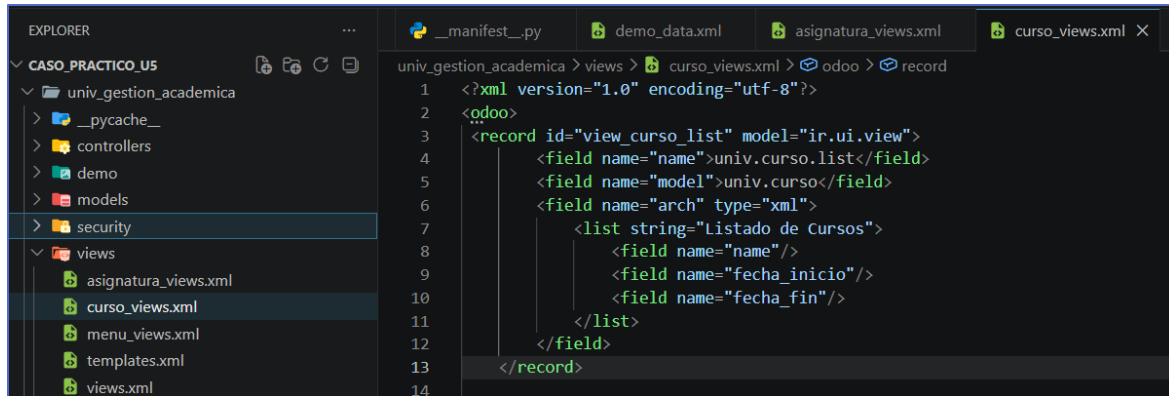
La lógica visual de Odoo va un poco al revés: primero creas la "Acción" (qué pasa cuando haces clic) y luego el "Menú" (el botón que ves en pantalla).

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3      <record id="action_univ_curso" model="ir.actions.act_window">
4          <field name="name">Cursos</field>
5          <field name="res_model">univ.curso</field>
6          <field name="view_mode">list,form</field>
7      </record>
8
9      <record id="action_univ_asignatura" model="ir.actions.act_window">
10         <field name="name">Asignaturas</field>
11         <field name="res_model">univ.asignatura</field>
12         <field name="view_mode">list,form</field>
13     </record>
14
15     <menuitem id="menu_univ_root" name="Universidad" sequence="10"/>
16
17     <menuitem id="menu_univ_gestion" name="Gestión Académica"
18         parent="menu_univ_root" sequence="10"/>
19
20     <menuitem id="menu_cursos" name="Cursos Activos"
21         parent="menu_univ_gestion" action="action_univ_curso" sequence="1"/>
22     <menuitem id="menu_asignaturas" name="Catálogo de Asignaturas"
23         parent="menu_univ_gestion" action="action_univ_asignatura" sequence="2"/>
24 </odoo>
  
```

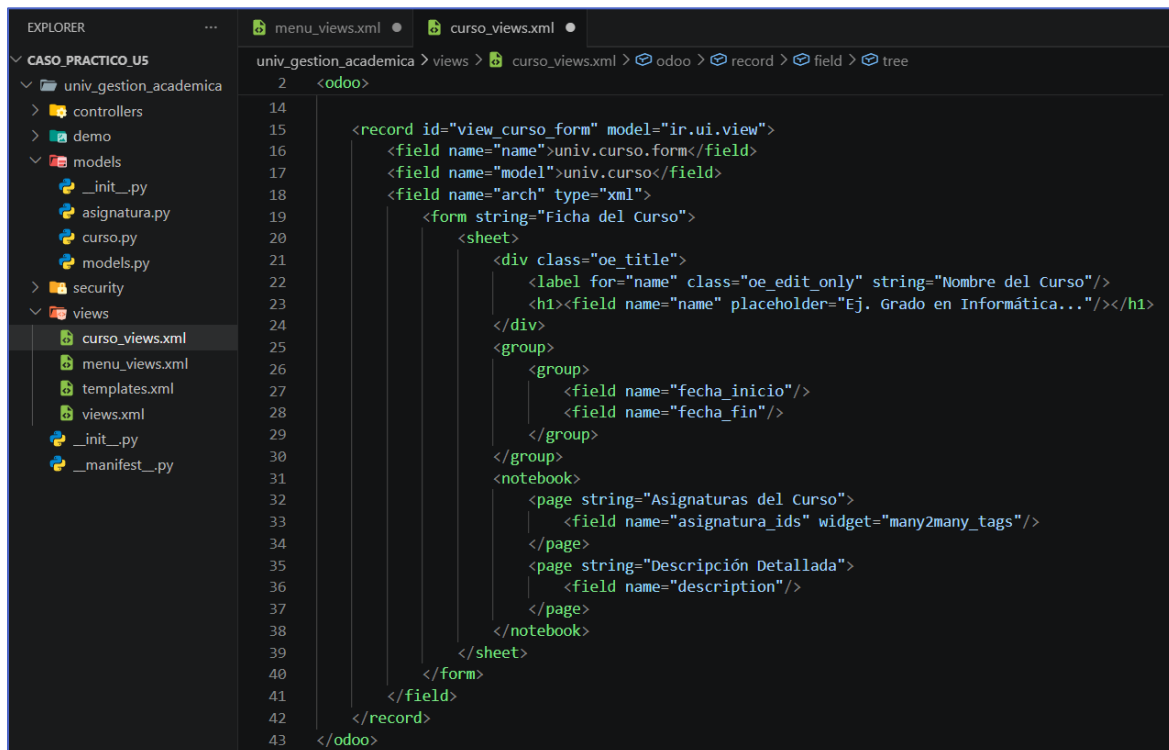
4.2. Vistas del Curso (views/curso_views.xml)

Aquí es donde definimos el listado general (tree) y la vista detalle de cada curso (form).



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The 'views' folder is expanded, showing 'curso_views.xml' selected. The main editor displays the XML code for the 'view_curso_list' record. The code defines a list view for 'univ.curso.list' with fields for 'name', 'fecha_inicio', and 'fecha_fin'.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3   <record id="view_curso_list" model="ir.ui.view">
4     <field name="name">univ.curso.list</field>
5     <field name="model">univ.curso</field>
6     <field name="arch" type="xml">
7       <list string="Listado de Cursos">
8         <field name="name"/>
9         <field name="fecha_inicio"/>
10        <field name="fecha_fin"/>
11      </list>
12    </field>
13  </record>
14 </odoo>
```

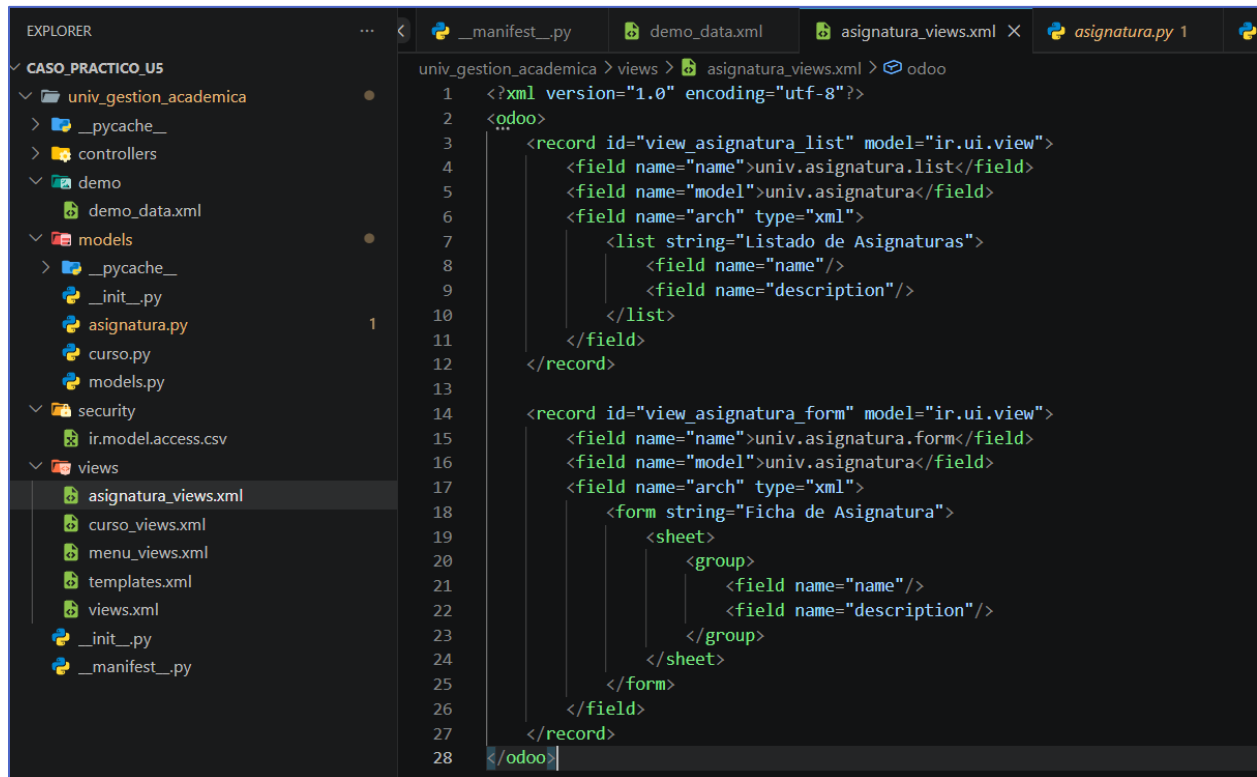


The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The 'views' folder is expanded, showing 'curso_views.xml' selected. The main editor displays the XML code for the 'view_curso_form' record. The code defines a form view for 'univ.curso.form' with fields for 'name', 'fecha_inicio', 'fecha_fin', and a 'description' field. It also includes a 'notebook' widget for displaying 'Asignaturas del Curso' and 'Descripción Detallada'.

```
2 <odoo>
14
15   <record id="view_curso_form" model="ir.ui.view">
16     <field name="name">univ.curso.form</field>
17     <field name="model">univ.curso</field>
18     <field name="arch" type="xml">
19       <form string="Ficha del Curso">
20         <sheet>
21           <div class="oe_title">
22             <label for="name" class="oe_edit_only" string="Nombre del Curso"/>
23             <h1><field name="name" placeholder="Ej. Grado en Informática..."/></h1>
24           </div>
25           <group>
26             <group>
27               <field name="fecha_inicio"/>
28               <field name="fecha_fin"/>
29             </group>
30           </group>
31           <notebook>
32             <page string="Asignaturas del Curso">
33               <field name="asignatura_ids" widget="many2many_tags"/>
34             </page>
35             <page string="Descripción Detallada">
36               <field name="description"/>
37             </page>
38           </notebook>
39         </sheet>
40       </form>
41     </field>
42   </record>
43 </odoo>
```


4.3. Vistas de las asignaturas (views/asignatura views.xml)

Aquí es donde definimos el listado general (tree) y la vista detalle de cada asignatura (form).



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <record id="view_asignatura_list" model="ir.ui.view">
4         <field name="name">univ.asignatura.list</field>
5         <field name="model">univ.asignatura</field>
6         <field name="arch" type="xml">
7             <list string="Listado de Asignaturas">
8                 <field name="name"/>
9                 <field name="description"/>
10            </list>
11        </field>
12    </record>
13
14    <record id="view_asignatura_form" model="ir.ui.view">
15        <field name="name">univ.asignatura.form</field>
16        <field name="model">univ.asignatura</field>
17        <field name="arch" type="xml">
18            <form string="Ficha de Asignatura">
19                <sheet>
20                    <group>
21                        <field name="name"/>
22                        <field name="description"/>
23                    </group>
24                </sheet>
25            </form>
26        </field>
27    </record>
28 </odoo>
```

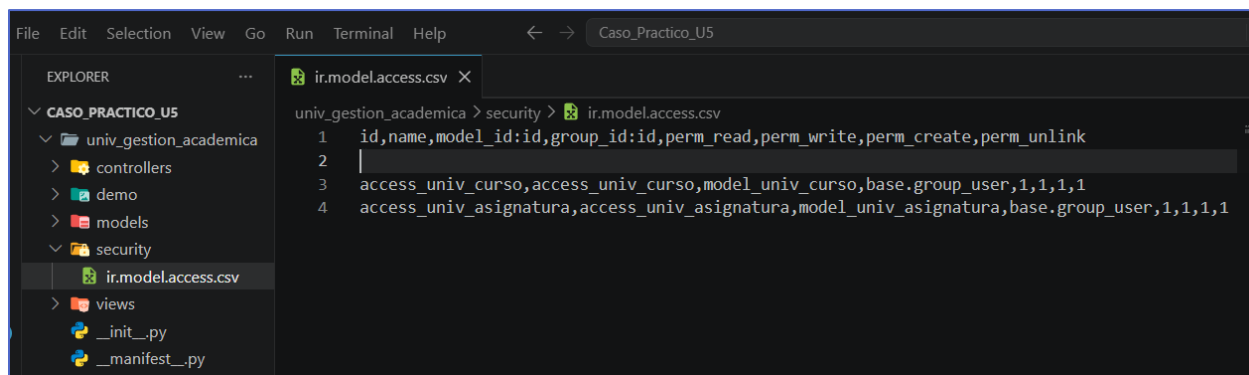
SEGURIDAD: CONTROL DE ACCESOS

(IR.MODEL.ACCESS.CSV)

En nuestras primeras pruebas nos llevamos el típico susto de novatos: instalamos el módulo, el servidor no dio ningún error, pero **al entrar a la aplicación los menús y las vistas no aparecían por ningún lado**. Mirando la documentación para ver qué habíamos roto, descubrimos que **Odoo**, por pura seguridad, viene "capado" de fábrica y **bloquea el acceso por defecto a cualquier modelo nuevo que te inventemos**. Para que alguien pueda ver o interactuar con esas tablas de la base de datos, **tenemos que decirle al sistema explícitamente quién tiene permiso**.

En vez de liarnos a crear la estructura de permisos desde cero, **aprovechamos los archivos que nos había autogenerado** el comando scaffold al principio y nos fuimos directos a la plantilla **security/ir.model.access.csv**.

Lo que hicimos fue limpiar un poco las líneas de ejemplo que traía comentadas y declarar los permisos de nuestros dos modelos jugando con ceros y unos (1 para permitir, 0 para bloquear). Básicamente, les dimos permisos totales de Lectura (perm_read), Escritura (perm_write), Creación (perm_create) y Borrado (perm_unlink) al grupo de usuarios internos que trae el sistema por defecto (base.group_user). El archivo nos quedó así:



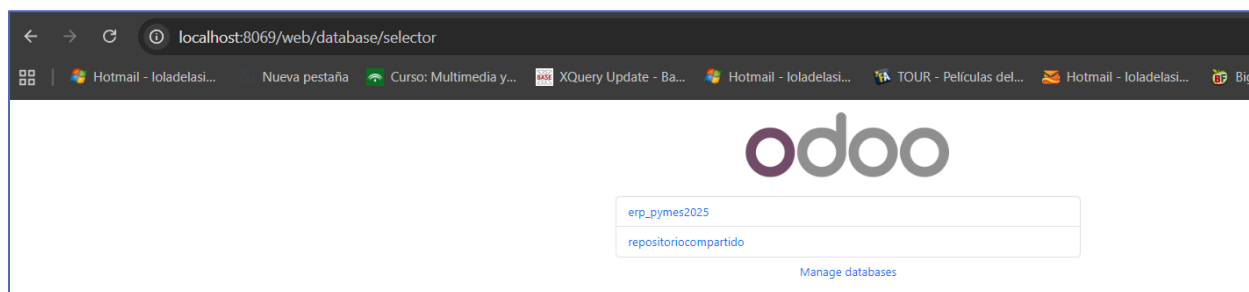
```
univ_gestion_academica > security > ir.model.access.csv
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 |
3 access_univ_curso,access_univ_curso,model_univ_curso,base.group_user,1,1,1,1
4 access_univ_asignatura,access_univ_asignatura,model_univ_asignatura,base.group_user,1,1,1,1
```

PROCESO DE INSTALACIÓN Y VALIDACIÓN DEL MÓDULO

Una vez que ya teníamos toda la lógica en Python, montadas las vistas en XML y habíamos configurado el puente en el archivo `odoo.conf`, el siguiente paso era **hacer que el módulo apareciera por fin en la interfaz web de Odoo**. Acostumbrados a darle al *play* en el IDE y que todo arranque solo, aquí nos dimos cuenta de que hay que seguir un ritual bastante estricto.

6.1. Acceso al Entorno y Gestión de la Base de Datos

Como en Odoo no ejecutamos un programa de escritorio sino que atacamos a una base de datos a través del navegador, lo primero fue entrar a <http://localhost:8069>.



A la hora de elegir la base de datos, nuestra recomendación técnica (y lo que nos ahorró bastantes dolores de cabeza) es **crear una base de datos totalmente nueva** solo para esta práctica. Al crearla, es vital acordarse de no marcar la casilla de cargar datos de prueba (**Demo data**). Nosotros crearemos los datos de entrada para probar el módulo. Una vez creada, entramos con el clásico admin / admin.

Para crear nuestro propio Demo Data hicimos los siguientes cambios en nuestros archivos.

manifest .py

```
ir.model.access.csv  _manifest_.py  demo.xml  curso.py 1
univ_gestion_academica > _manifest_.py
10     - Relación estructurada N:M (Muchos a Muchos) entre cursos y asignaturas.
11     - Generación automática de informes en formato PDF (QWeb) con el plan de estudios.
12     "",
13     'author': "Departamento de Informática - Equipo de Desarrollo",
14     'category': 'Education',
15     'version': '1.0',
16
17     # Dependencias base del sistema ERP
18     'depends': ['base'],
19
20     # Archivos que Odoo debe cargar (El orden de declaración es estricto)
21     'data': [
22         'security/ir.model.access.csv', # 1º: Permisos y accesos
23         'views/menu_views.xml',        # 2º: Estructura de menús principales
24         'views/asignatura_views.xml',   # 3º: Vistas del modelo Asignatura
25         'views/curso_views.xml',       # 4º: Vistas del modelo Curso
26         'reports/curso_report.xml',    # 5º: Plantillas de impresión QWeb
27
28         # Movemos el archivo de demo aquí para que Odoo lo instale obligatoriamente
29         'demo/demo_data.xml',
30     ],
31
32     # Datos inyectados automáticamente para realizar pruebas funcionales
33     'demo': [
34
35     ],
```

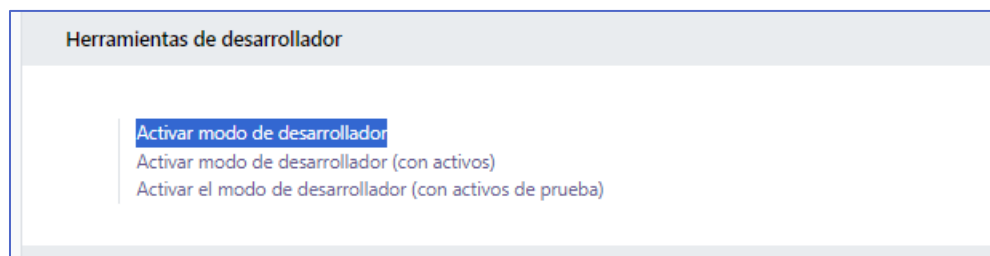
demo data-xml

```
File Edit Selection View Go Run Terminal Help
univ_gestion_academica > demo > demo_data.xml > odoo > data > record
1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3  <data>
4  <record id="demo_asignatura_bbdd" model="univ.asignatura">
5      <field name="name">Bases de Datos</field>
6      <field name="description">Introducción a bases de datos
7      relacionales y lenguaje SQL.</field>
8  </record>
9  <record id="demo_asignatura_prog" model="univ.asignatura">
10     <field name="name">Programación</field>
11     <field name="description">Fundamentos de programación orientada a
12     objetos y estructuras de datos.</field>
13 </record>
14 <record id="demo_curso_dam" model="univ.curso">
15     <field name="name">Ciclo Superior DAM</field>
16     <field name="description"><![CDATA[<p><strong>Desarrollo de
17     Aplicaciones Multiplataforma</strong>. Programa oficial de
18     formación profesional superior.</p>]]></field>
19     <field name="fecha_inicio">2026-09-15</field>
20     <field name="fecha_fin">2027-06-20</field>
21     <field name="asignatura_ids" eval="[(6, 0, [ref
22     ('demo_asignatura_bbdd'), ref('demo_asignatura_prog')])]" />
23 </record>
24 </data>
25 </odoo>
```

6.2. Activación del Modo Desarrollador (Paso Imprescindible)

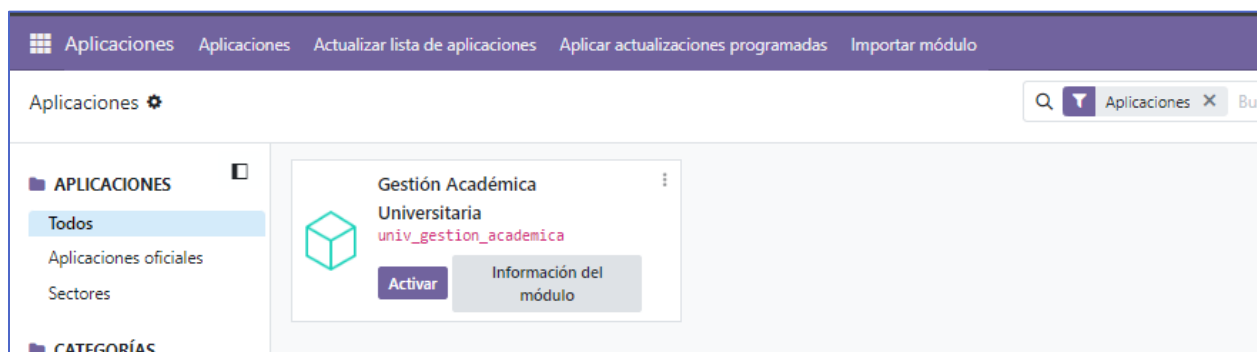
Odoo le oculta todas las opciones técnicas a los usuarios normales. Si intentas buscar tu módulo recién creado así, no lo vas a encontrar. Tienes que convertirte en "**superusuario**". Para hacerlo por las buenas, te vas al menú de Ajustes (**Settings**), bajas hasta el final del todo y le das al enlace de "**Activar modo desarrollador**". Sabes que lo has hecho bien porque te aparece el icono de un escarabajo (un *bug*) en la barra de arriba.

El truco que acabamos usando siempre: La forma rápida de hacer esto sin dar tantas vueltas es irte a la URL del navegador y plantarle un `?debug=1` justo después de `/web`.



6.3. Actualización de la Lista de Aplicaciones

Vale, nuestra **carpeta univ_gestion_academica** ya estaba en el disco duro, pero Odoo seguía sin saber que existía. Hay que forzar al sistema a que escanee la ruta nueva que le pusimos en el **addons_path**. Para eso, entramos en el menú principal de Aplicaciones y en la barra superior le dimos a "**Actualizar lista de aplicaciones**". Al confirmar en el cuadro de diálogo, es cuando Odoo va físicamente a nuestra carpeta, lee el archivo `__manifest__.py` y por fin registra el módulo en su catálogo interno.



6.4. Localización e Instalación del Módulo

Con el catálogo actualizado, fuimos a la barra de búsqueda de Aplicaciones y tecleamos nuestro nombre técnico (**univ_gestion_academica**). Ahí apareció por fin la "tarjeta" de nuestro módulo. Le dimos al botón de "Instalar" y el sistema se quedó pensando unos segundos. En ese rato es cuando Odoo está traduciendo nuestro código de Python para crear las tablas en PostgreSQL e inyectando nuestras vistas XML.



6.5. Verificación de Menús y Datos

Con la instalación terminada, tocaba comprobar si el esfuerzo había valido la pena: Abrimos el menú principal (el icono de los cuadraditos de arriba a la izquierda) y, efectivamente, ahí estaba nuestra nueva aplicación llamada "**Universidad**". Al meternos dentro, verificamos que en la barra de arriba salía el menú "**Gestión Académica**" con sus dos submenús colgando: "**Cursos**" y "**Asignaturas**".

Lo mejor fue entrar a la sección de "**Cursos**" y ver que ya nos aparecía un registro creado ("**Ciclo Superior DAM**"). Esto nos confirmó que el archivo **demo_data.xml** que nos habíamos peleado en configurar había inyectado los datos de prueba a la primera y que las relaciones entre las tablas funcionaban perfectamente.



07

GENERACIÓN DE INFORMES DINÁMICOS (QWEB)

Como nos pedían poder sacar en PDF la ficha de los cursos con sus asignaturas, nos tocó investigar **QWeb**, que es el **motor de plantillas** que trae **Odoo**. En el archivo **reports/curso_report.xml** montamos el diseño. Es una mezcla de HTML de toda la vida pero metiéndole variables dinámicas de Python usando etiquetas como **t-field** (para imprimir un dato) y **t-foreach** (para montar bucles).

```

EXPLORER
CASO_PRACTICO_US
  univ_gestion_academica
    __pycache__
    controllers
    demo
    models
    reports
      curso_report.xml
    security
    views
    __init__.py

univ_gestion_academica > reports > curso_report.xml > odoo
1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3
4      <record id="action_report_curso" model="ir.actions.report">
5          <field name="name">Imprimir Ficha del Curso</field>
6          <field name="model">univ.curso</field>
7          <field name="report_type">qweb-pdf</field>
8          <field name="report_name">univ_gestion_academica.report_curso_template</field>
9          <field name="report_file">univ_gestion_academica.report_curso_template</field>
10         <field name="binding_model_id" ref="model_univ_curso"/>
11         <field name="binding_type">report</field>
12     </record>

```

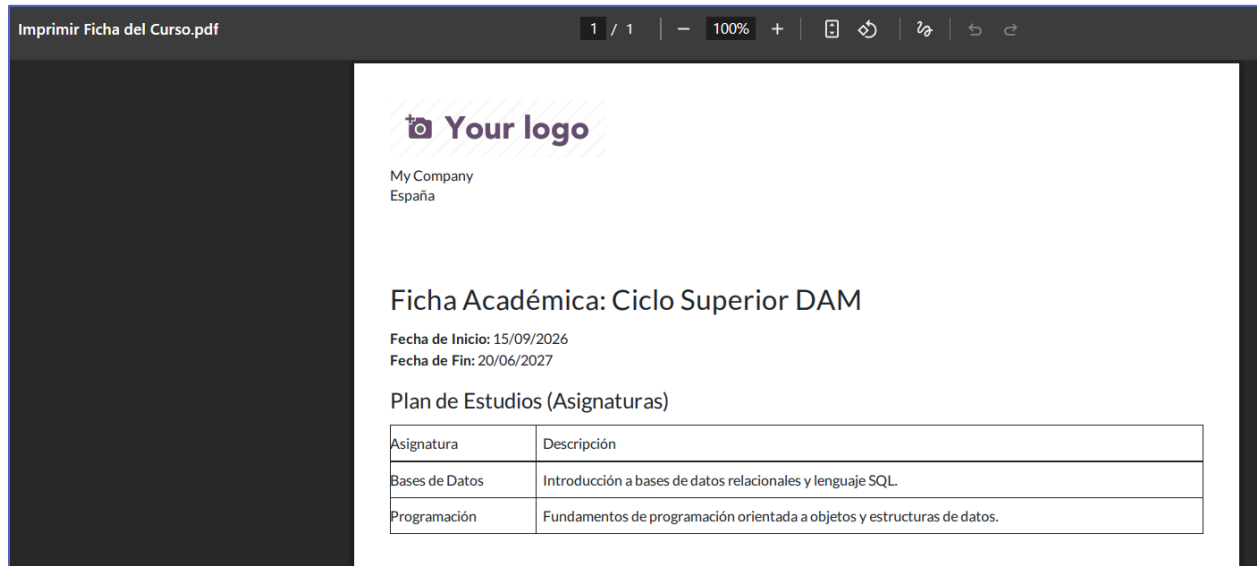
```

EXPLORER
CASO_PRACTICO_US
  univ_gestion_academica
    __pycache__
    controllers
    demo
    models
    reports
      curso_report.xml
    security
    views
    __init__.py
    __manifest__.py

univ_gestion_academica > reports > curso_report.xml > odoo
2  <odoo>
13
14      <template id="report_curso_template">
15          <t t-call="web.html_container">
16              <t t-foreach="docs" t-as="doc">
17                  <t t-call="web.external_layout">
18                      <div class="page">
19                          <h2>Ficha Académica: <span t-field="doc.name"/></h2>
20
21                          <div><strong>Fecha de Inicio:</strong> <span t-field="doc.fecha_inicio"/></div>
22                          <div><strong>Fecha de Fin:</strong> <span t-field="doc.fecha_fin"/></div>
23
24                          <h4 style="margin-top: 20px;">Plan de Estudios (Asignaturas)</h4>
25                          <table class="table table-bordered">
26                              <thead>
27                                  <tr>
28                                      <th>Asignatura</th>
29                                      <th>Descripción</th>
30                                  </tr>
31                              </thead>
32                              <tbody>
33                                  <tr t-foreach="doc.asignatura_ids" t-as="asig">
34                                      <td><span t-field="asig.name"/></td>
35                                      <td><span t-field="asig.description"/></td>
36                                  </tr>
37                              </tbody>
38                          </table>
39                      </div>
40                  </t>
41              </t>

```

Resultado del informe creado impreso:



DEPURACIÓN, TRATAMIENTO DE ERRORES E INCIDENCIAS EN ENTORNO REAL

Como suele pasar cuando te metes con el despliegue de soluciones empresariales complejas, el proceso de instalación y validación no estuvo libre de baches. Lejos de intentar arreglar las cosas "a ojo" o por pura intuición, nos tocó aplicar metodologías reales de análisis de registros (*logs*), descifrar las trazas de error (*tracebacks*) y depurar el código directamente desde la terminal de VS Code. Aquí detallamos las cuatro incidencias críticas que nos encontramos y las soluciones técnicas que aplicamos para estabilizar el sistema:

8.1. Conflicto Crítico de Versiones en el Motor de Base de Datos (PostgreSQL)

- **El Problema:** Al intentar crear la base de datos marcando la casilla de los datos de demostración globales de Odoo (*Load demo data*), la interfaz web se caía por completo y nos soltaba de forma persistente el temido error HTTP 500 (*Internal Server Error*).
- **Depuración e Identificación:** Al ir a mirar la consola activa de VS Code donde estaba corriendo el servidor de Odoo, localizamos este aviso del sistema:

UserWarning: Postgres version is 120004, lower than minimum required 130000

- **Causa Raíz:** Odoo 19 mete dependencias y funciones de consulta bastante avanzadas en sus datos de prueba de fábrica que exigen obligatoriamente PostgreSQL 13 o superior. Al estar corriendo nuestro servidor local sobre PostgreSQL 12, el proceso de creación de las tablas base abortaba a mitad de camino, dejando la base de datos corrupta o en estado "zombi". Además, el navegador guardaba una *cookie* residual (*session_id*) que nos metía en un bucle infinito volviendo a cargar el error 500.
- **Resolución Técnica:** Montamos un protocolo de tres pasos para aislar el fallo:
 1. Abrimos el navegador en modo Incógnito para saltarnos las *cookies* corruptas de la sesión y poder entrar limpios al gestor de bases de datos (/web/database/manager).
 2. Borramos del todo la base de datos que se había quedado a medias.
 3. Creamos una base de datos nueva desde cero desmarcando la casilla de los datos demo de Odoo. Para no perder nuestros propios datos de prueba sin tener que actualizar todo el motor de PostgreSQL del ordenador, lo que hicimos fue reconfigurar el archivo `__manifest__.py`, metiendo la ruta `demo/demo_data.xml` dentro del diccionario obligatorio 'data'. Así forzamos a Odoo a cargarlos como registros normales del módulo, esquivando el código conflictivo de fábrica.

8.2. Error de Inyección por Referencia Circular (ImportError)

- **El Problema:** El servidor se paraba en seco nada más arrancar y leer el módulo, lo que hacía que Odoo ni siquiera mostrara nuestra aplicación en el catálogo.
- **Depuración e Identificación:** La traza de error (*traceback*) que nos escupía la terminal de Python decía esto:

ImportError: cannot import name 'asignatura' from partially initialized module

- **Causa Raíz:** Nos equivocamos al organizar la arquitectura del módulo. Habíamos metido todo el bloque de importación del ORM directamente en el archivo descriptor inicial (`__init__.py` de la raíz del proyecto). Python se volvía loco buscando los scripts de los modelos en la carpeta raíz en lugar de mirar en la subcarpeta que le correspondía.
- **Resolución Técnica:** Reestructuramos el orden de las llamadas del intérprete:
 1. Dejamos el `__init__.py` principal de la raíz súper simple, solo para delegar la carga al subdirectorio: `from . import models`.
 2. Creamos un segundo archivo `__init__.py` exclusivo dentro de la carpeta `/models` para indexar de forma aislada nuestros archivos reales: `from . import asignatura` y `from . import curso`.

8.3. Depreciación Sintáctica y Cambio de Estándar en Vistas (Odoo 19)

- **El Problema:** Al darle al botón de "Instalar" el módulo, el motor del *framework* cancelaba el proceso y nos lanzaba un aviso técnico en la web.
- **Depuración e Identificación:** Al activar el modo desarrollador (con el truco del ?debug=1 en la URL), pudimos ver el error de sintaxis real que estaba oculto:

odoo.tools.convert.ParseError: Invalid view type: 'tree'. Allowed types are: list, form...

- **Causa Raíz:** Al programar las vistas nos guiamos por apuntes y tutoriales de versiones anteriores (como Odoo 16 o 17), donde se usaba la etiqueta clásica <tree> para pintar las listas de asignaturas y cursos. Sin embargo, en el núcleo de Odoo 19, la etiqueta tree se ha eliminado por completo del *framework*, pasando a ser obligatorio usar la nomenclatura <list>.
- **Resolución Técnica:** Nos tocó revisar y actualizar los archivos asignatura_views.xml, curso_views.xml y las acciones de ventana en menu_views.xml. Cambiamos todas las aperturas y cierres de <tree>...</tree> por <list>...</list>, y modificamos los modos de visualización en las acciones para dejarlo como view_mode = "list,form".

8.4. Desajuste de Integridad Referencial de Datos de Entrada (ValueError)

- **El Problema:** Saltaba un error de validación bastante feo justo al cargar los registros automáticos de prueba en la base de datos.
- **Depuración e Identificación:** El analizador XML del ORM nos devolvió esta excepción:

ValueError: Invalid field 'creditos' in 'univ.asignatura'

- **Causa Raíz:** Había una falta de coincidencia total entre la capa de datos (Python) y la capa de inicialización (XML). En el archivo de datos demo intentábamos meter valores en una columna llamada creditos, pero al ir a mirar el archivo real asignatura.py, vimos que ese modelo solo tenía los campos name y description. Estábamos intentando guardar datos en una columna que no existía.
- **Resolución Técnica:** Limpiamos el archivo demo_data.xml borrando el campo inexistente para adaptarlo a la estructura real del modelo en Python. Para el nuevo registro de prueba, aprovechamos el campo description para meter los textos explicativos y usamos la tupla de Many2many [(6, 0, [lista_de_ids])] de forma limpia para asociar el curso con sus asignaturas sin romper las restricciones de la base de datos.

CONCLUSIONES

Llegar al final de este caso práctico ha supuesto una inmersión tremenda y muy realista en el ecosistema de Odoo. A nivel funcional, el módulo "Gestión Académica Universitaria" hace exactamente lo que se pedía: te deja registrar los cursos y las asignaturas, cruza la información de forma transparente gracias al ORM, y saca informes en PDF que tienen un aspecto súper profesional y adaptado al diseño del sistema.

Si lo miramos desde el lado técnico, la práctica nos ha servido para que por fin nos haga "clic" en la cabeza la arquitectura MVC tan particular que usa Odoo. Ahora entendemos perfectamente por qué el `__manifest__.py` es como el contrato de la aplicación, y lo vitales que son los archivos `__init__.py` para que el intérprete de Python no se deje modelos sin cargar. Además, usar la versión 19 nos ha forzado a actualizarnos y darnos cuenta de cómo evoluciona el *framework*; un buen ejemplo de esto fue el susto que nos llevamos al ver que las etiquetas clásicas `<tree>` ya no valían para las vistas y tuvimos que migrar todo a `<list>`.

De todas formas, el mayor aprendizaje de este proyecto no ha sido aprendernos la sintaxis de memoria, sino coger soltura depurando. Enfrentarnos a problemas de despliegue reales (como que la versión de PostgreSQL no fuera compatible, los fallos de carga o que el XML no coincidiera con los campos de Python) nos ha quitado la mala costumbre de programar por ensayo-error. Aprender a pararse, leer los *logs*, descifrar el *traceback* y aplicar la solución exacta (como tocar el *addons_path* o limpiar las *cookies* del navegador) es lo que de verdad nos ha enseñado cómo se trabaja profesionalmente en el desarrollo de ERPs.

REFERENCIAS

<https://www.odoo.com/documentation/master/developer.html>

<https://www.odoo.com/documentation/master/developer/reference/backend/oracle.html>

https://www.odoo.com/documentation/master/developer/tutorials/define_module_data.html

https://www.odoo.com/documentation/19.0/developer/reference/user_interface/view_records.html

<https://www.odoo.com/documentation/19.0/developer/reference/backend/reports.html>

<https://docs.python.org/3/tutorial/modules.html>

<https://www.postgresql.org/docs/>

How to Install Odoo 18 with Pycharm | Setup Odoo 18 Development Environment Using Pycharm

https://www.youtube.com/watch?v=aMDwueq9_sY&list=PLeJtXzTubzj8jrdV7luM1G_xfUanwbKh

How To Create Module In Odoo 17 | Odoo 17 Developement

<https://www.youtube.com/watch?v=kxbPgOLkNxw>